Functional dependency

In R, X --> Y iff for any r1, r2 in R, r1.X=r2.X implies r1.Y=r2.Y.

Superkey

In R, X is a superkey iff for any Y subset of attributes of R, X --> Y

Key

In R, X is a key iff (a) X is a superkey and (b) X is minimal

Armstrong axioms (AA)

If X includes Y, then X --> Y (reflexivity)
If X --> Y, then XZ --> YZ (augmentation)
X --> Y and Y --> Z implies X --> Z (transitivity)

Extra rules

- X --> YZ implies X --> Y and X --> Z (decomposition)
- X --> Y and X --> Z implies X --> YZ (union)

Closure of a set of FDs

Given a set of FDs F, the closure of F, denoted F^+, is the set of all FDs that can be derived from F via AA

Boyce-Codd normal form (BCNF)

Relation R is in BCNF iff for any dependency X --> A: A is in X, or X is a superkey

Third normal form (3NF)

Relation R is in BCNF iff for any dependency X --> A: A is in X, or X is a superkey, or A is part of a key

BCNF implies 3NF

Lossless-join decomposition

Decomposing R into R1 and R2 is lossless-join iff for any instances r, r1, r2, r1 join r2 = r.

Theorem (criterium for LJD)

If (R1 intersect R2) --> R1, or (R1 intersect R2) --> R2, then decomposing R into R1 and R2 is LJ.

Dependency restriction

Let R1 be a subset of the attributes of R.
F_{R1} is the restriction of F^+ to the attributes of R1.

Dependency-preserving decomposition

A decomposition of R into R1, R2, ... Rn preserves dependencies if
(F_{R1} U F_{R2} U... F_{Rn})$^+$= F$^+$

Decomposition into BCNF

Start with F^+, repeat:

- If X --> Y violates BCNF
- Then decompose R into (XY), (R - Y).
- Repeat, until every relation is in BCNF.

This is guaranteed to be lossless-join.
In general, there may not exist a BCNF decomposition that preserves dependencies.

Decomposition into 3NF (1)

1. Like for BCNF but stop when all relations in 3NF (earlier).
2. To ensure DP, for any dependency X --> Y not covered, add (X, Y)
    - This may violate 3NF! So, no solution yet

Minimal cover

G is a minimal cover if it contains any dependency X --> A that holds in F^+ (where A is a single attribute) and G is minimal.

Decomposition into 3NF (2)

1. Decompose using G until no 3NF-violating dependency remains.
2. For any G dependency X --> A not covered, add a relation (XA):

- X is a key in this relation;
- Any other G dependency that holds over (XA), e.g., X1 --> A1 for some A, will have A1 part of X, and X is a key.
  Thus, such a dependency does not violate 3NF.

Thus: **there exists a LJ, DP decomposition of any relation in 3NF** using G.